Week 2 Day 1:

Task 1: Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

Context:

As a Test lead generate me a test plan for payment module validation with only UPI and IMPS Payment modes Instruction

-ASK me 5 questions one by one for test plan

-Ask me the next question after me providing the answer

-Test plan should be following the industry standard

By interacting with Qwen Chat, prepared a test plan in excel format.

TP-PAY-UPI-IMPS-001_Payment_Module_Test_Plan

Prompt written using ICEPOT Technique:

Context:

As an Expertise QA Automation Architect with BFSI domain knowledge, you are required to generate an Automation Test Plan in Excel-ready table format.

Instructions:

Generate Automation Test Plan for Payment (Fund Transfer) module

Generate ONLY for UPI and IMPS

[IMPORTANT] Include ONLY Functional Automation scope (UI + API where applicable)

[CRITICAL] Focus ONLY on Happy Path End-to-End flows

[IMPORTANT] Use Selenium with Java for UI automation

[IMPORTANT] Use Jenkins for CI/CD

[IMPORTANT] Output must be Excel-ready (comma-separated table format)

[IMPORTANT] Include all sections in structured row-wise format

[IMPORTANT] Avoid manual testing scope completely

Examples:

1. Example Entry:
{

"section": "Scope",
"subSection": "In Scope",
"details": "UPI fund transfer via UPI ID"
}
2. Example Strategy:
{
"section": "Test Strategy",
"subSection": "",
"details": "Hybrid automation approach using UI and API"
}
3. Example Risk:
{
"section": "Risk & Mitigation",
"subSection": "Risk",
"details": "Unstable environment"
}

{
"section": "Risk & Mitigation",
"subSection": "Mitigation",
"details": "Retry mechanism implemented"
}

Persona:

Review the generated test plan as a Domain-Expertise QA Team Lead ensuring completeness, correctness, and automation feasibility

Output Format:
[STRICT] Generate output in JSON format ONLY
[STRICT] Do NOT include any explanation or extra text
[STRICT] Output must be a JSON ARRAY
[STRICT] Each object must represent one row/entry
[STRICT] Do NOT merge multiple points into one object
[STRICT] Maintain consistent keys across all objects

JSON Output Format
[
{
"section": "Test Plan Identifier",
"subSection": "",
"details": "ATP_PAYMENTS_UPI_IMPS_001"

},
{
"section": "Objective",
"subSection": "",
"details": "To design automation framework for UPI and IMPS payment flows"
}
]

Sections to Cover (MANDATORY):
- Test Plan Identifier
- Objective
- Scope (In Scope, Out of Scope)
- Test Strategy
- Test Environment
- Tools & Technologies
- Test Data Strategy
- Entry Criteria
- Exit Criteria
- Test Deliverables
- Risk & Mitigation (Risk, Mitigation)
- Automation Framework Approach
- Execution Plan
- Reporting & Monitoring
- Assumptions & Dependencies

Tone:

Professional and implementation-ready

Task 2: Design the automation framework structure using a chain-of-thought approach

Prompt:

Context:

- As an Automation Architect, generate me a playwright with Typescript framework structure Instruction

-ASK me 5 questions one by one for framework creation

-Ask me the next question after me providing the answer

-Framework should be Enterprise following the industry standard

- Review the framework generated as an Automation Lead and provide comments in TL: DR

Framework details added in the document: Playwright_Framework_Document

Review comments:

✅ **What's Good**

- Strong layered architecture (UI + API + Core)
- Hybrid testing approach (API + UI) is solid
- CI/CD, parallel, cross-browser setup is enterprise-ready
- Security (Key Vault) handled well
- Reporting + logging (Allure, screenshots, videos) is good

⚠️ **Key Risks**

- ❌ **BDD + Playwright combo** → can cause maintenance & execution issues
- ❌ **No strong fixture design** → impacts scalability
- ❌ **Weak test data strategy** → risk of flaky tests
- ❌ **No API schema validation** → not enough for BFSI
- ❌ **Missing tagging strategy** → poor test control
- ❌ **Jira/Zephyr integration not fully defined**

🔧 **Must Fix (High Priority)**

1. Fix **BDD vs Playwright runner conflict**
2. Implement **robust fixture architecture**
3. Strengthen **test data management & cleanup**
4. Add **API schema validation (AJV)**
5. Define **test tagging strategy (@smoke, @regression, etc.)**

📊 **Final Verdict**

👉 **Good foundation (75% ready)**
👉 Needs **stability + governance layers** to be truly enterprise-grade